

Lab 3 - Memory Safety Report

Task 1 - Solution

1. Program Code (program1.c)

```
#include<stdio.h>
#include<stdlib.h>

void program1(int N) {
    int *arr = malloc(N); // Problematic allocation
    for(int i = 0; i < N; i++) {
        arr[i] = i * i;
        printf("arr[%d] = %d\n", i, arr[i]);
    }
}

int main() {
    program1(4);
}
```

2. Compilation & Execution

```
# Compile with debugging symbols
gcc -Wall -Werror -g -std=c99 -o program1 -O0 program1.c

# Run normally
./program1
```

Output:

```
arr[0] = 0
arr[1] = 1
arr[2] = 4
arr[3] = 9
```

Program appears to work normally but contains hidden bugs (we are booking N bytes we need N* (size of the var))

3. Valgrind Analysis

```
valgrind --leak-check=full ./program1
```

Key Valgrind Findings:

Invalid Write Errors (3 occurrences)

```
==51351== Invalid write of size 4
==51351==    at 0x1091AC: program1 (program1.c:7)
Cause: Writing to arr[1], arr[2], and arr[3] in a 4-byte buffer (only space
for 1 int)
```

Invalid Read Errors (3 occurrences)

```
==51351== Invalid read of size 4
==51351==    at 0x1091C2: program1 (program1.c:8)
Cause: Reading from beyond allocated memory
```

Memory Leak

```
==51351== 4 bytes in 1 blocks are definitely lost
Cause: Missing free(arr)
```

4. Problem Diagnosis Root Cause:

```
int *arr = malloc(N); // Allocates N bytes instead of N integers
System Specs:
```

```
sizeof(int) = 4 bytes (typical 64-bit systems)
```

Needed: $4 \times 4 = 16$ bytes

Allocated: 4 bytes

Memory Layout Visualization:

```
Allocated memory: [0][ ][ ][ ] (4 bytes)
Actual usage:    [0][1][4][9] (16 bytes)
                  ^ Buffer overflow!
```

5. CWE Identification CWE-787: Out-of-bounds Write

Writing to memory locations outside the allocated buffer

CWE-788: Access of Memory Location After End of Buffer

Both read and write operations beyond buffer limits

6. Fixed Code (program1_fixed.c)

```
#include<stdio.h>
#include<stdlib.h>

void program1(int N) {
    int *arr = malloc(N * sizeof(int)); // Correct allocation
    if(arr == NULL) {
        fprintf(stderr, "Memory allocation failed\n");
        return;
    }

    for(int i = 0; i < N; i++) {
        arr[i] = i * i;
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    free(arr); // Prevent memory leak
}

int main() {
    program1(4);
}
```

7. Verification Recompile and Test:

```
gcc -Wall -Werror -g -std=c99 -o program1_fixed program1_fixed.c
valgrind --leak-check=full ./program1_fixed
```

Clean Valgrind Output:

```
==51521== HEAP SUMMARY:
==51521==      in use at exit: 0 bytes in 0 blocks
==51521==    total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==51521==
==51521== All heap blocks were freed -- no leaks are possible
==51521== ERROR SUMMARY: 0 errors from 0 contexts
```

Key Lessons Memory Allocation:

- Always use `malloc(N * sizeof(datatype))`
- Check return value for NULL
- Valgrind Usage:
 - Essential for detecting hidden memory errors
 - Always run with `--leak-check=full`
- Memory Safety:
 - Buffer overflows can exist without immediate crashes
 - Undefined behavior != Safe behavior
- Secure Coding:
 - Always pair `malloc()` with `free()`
 - Use compiler flags (`-Wall -Werror`) for basic checks

The screenshot displays a desktop environment with a web browser and a terminal window.

Web Browser (Moodle LMS):

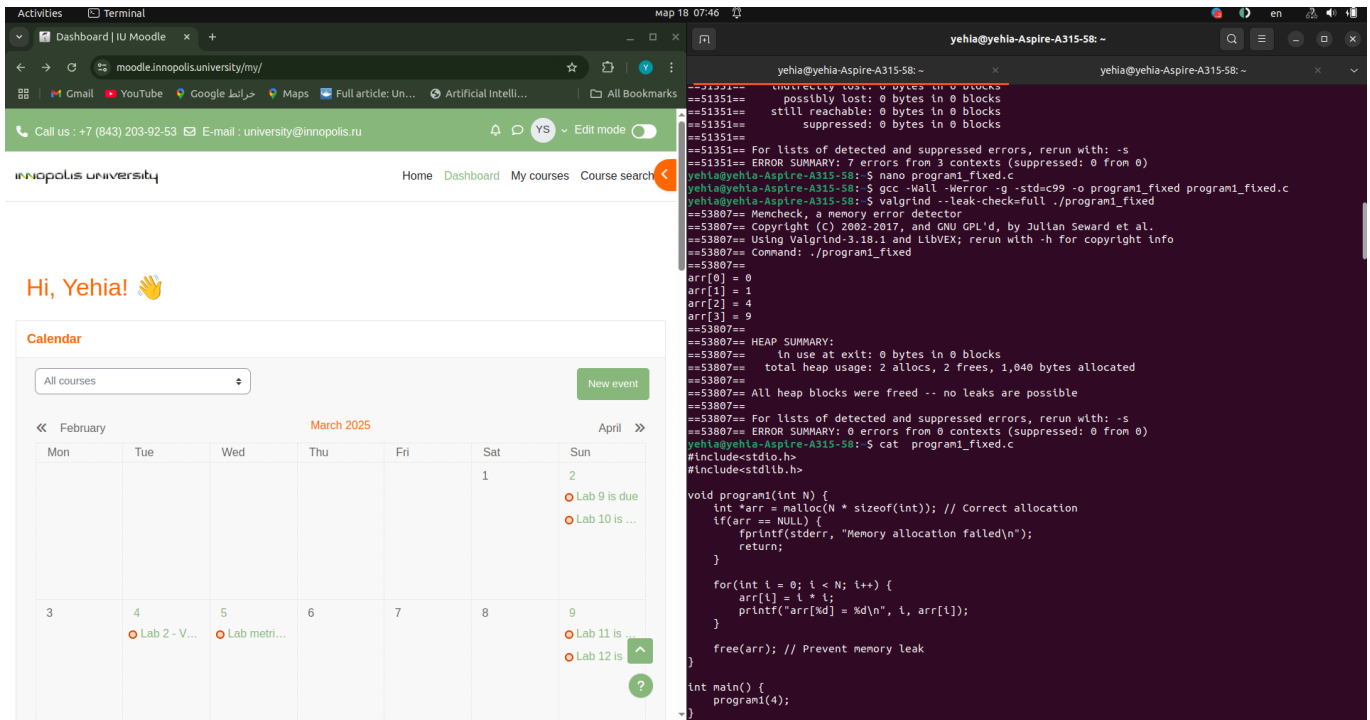
- Address bar: `moodle.innopolisuniversity/my/`
- Header: innopolis university, Home, Dashboard, My courses, Course search
- Greeting: Hi, Yehia! 🙌
- Calendar:
 - Month: March 2025
 - Days: Mon, Tue, Wed, Thu, Fri, Sat, Sun
 - Events:
 - Lab 2 - V...
 - Lab metri...
 - Lab 9 is due
 - Lab 10 is ...
 - Lab 11 is ...
 - Lab 12 is ...

Terminal Window:

```
yehia@yehia-Aspire-A315-S8: ~$ sudo apt-get update
sudo apt-get install gcc valgrind
[sudo] password for yehia:
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://ru.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 https://download.docker.com/linux/ubuntu jammy InRelease
Hit:4 https://apt.releases.hashicorp.com jammy InRelease
Hit:5 https://dl.google.com/linux/chrome/deb stable InRelease
Get:6 http://ru.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:7 http://ru.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:8 http://ru.archive.ubuntu.com/ubuntu jammy-updates/main amd64 DEP-11 Metadata [183 kB]
Get:9 http://ru.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 DEP-11 Metadata [212 B]
Get:10 http://ru.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 DEP-11 Metadata [359 kB]
Get:11 http://ru.archive.ubuntu.com/ubuntu jammy-updates/multiverse amd64 DEP-11 Metadata [940 B]
Get:12 http://ru.archive.ubuntu.com/ubuntu jammy-backports/main amd64 DEP-11 Metadata [7 028 B]
Get:13 http://ru.archive.ubuntu.com/ubuntu jammy-backports/restricted amd64 DEP-11 Metadata [212 B]
Get:14 http://ru.archive.ubuntu.com/ubuntu jammy-backports/universe amd64 DEP-11 Metadata [17,7 kB]
Get:15 http://ru.archive.ubuntu.com/ubuntu jammy-backports/multiverse amd64 DEP-11 Metadata [212 B]
Hit:16 http://packages.ros.org/ros2/ubuntu jammy InRelease
Fetched 744 kB in 1s (579 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
gcc is already the newest version (4:11.2.0-1ubuntu1).
valgrind is already the newest version (3.18.1-1ubuntu2).
The following packages were automatically installed and are no longer required:
  libflashrom1 libfido1-2 liblvm1 python3-docker python3-dockerpty python3-dockerpty python3-dotenv python
  3-jsonschema python3-pyrsistent python3-texttable python3-websocket
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
yehia@yehia-Aspire-A315-S8: ~$ nano program1.c
yehia@yehia-Aspire-A315-S8: ~$ cat program1.c
#include<stdio.h>
#include<stdlib.h>

void program1(int N) {
    int *arr = malloc(N);
    for(int i = 0; i < N; i++) {
        arr[i] = i * i;
        printf("arr[%d] = %d\n", i, arr[i]);
    }
}

int main() {
    program1(4); // should print the array [0, 1, 4, 9]
}
yehia@yehia-Aspire-A315-S8: ~$ gcc -Wall -Werror -g -std=c99 -o program1 -O0 program1.c
yehia@yehia-Aspire-A315-S8: ~$ ./program1
arr[0] = 0
arr[1] = 1
arr[2] = 4
arr[3] = 9
yehia@yehia-Aspire-A315-S8: ~$ valgrind --leak-check=full ./program1
==51351== Memcheck, a memory error detector
==51351== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==51351== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==51351== Command: ./program1
==51351==
==51351== Invalid write of size 4
==51351== at 0x1091AC: program1 (program1.c:7)
==51351== by 0x1091FE: main (program1.c:13)
==51351== Address 0x4aa8044 is 0 bytes after a block of size 4 alloc'd
==51351== at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==51351== by 0x109184: program1 (program1.c:5)
==51351== by 0x1091FE: main (program1.c:13)
==51351==
==51351== Invalid read of size 4
==51351== at 0x1091C2: program1 (program1.c:8)
==51351== by 0x1091FE: main (program1.c:13)
==51351== Address 0x4aa8044 is 0 bytes after a block of size 4 alloc'd
==51351== at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==51351== by 0x109184: program1 (program1.c:5)
==51351== by 0x1091FE: main (program1.c:13)
==51351==
arr[1] = 1
arr[2] = 4
arr[3] = 9
==51351==
==51351== HEAP SUMMARY:
==51351==    in use at exit: 4 bytes in 1 blocks
==51351== total heap usage: 2 allocs, 1 frees, 1,028 bytes allocated
==51351==
==51351== 4 bytes in 1 blocks are definitely lost in loss record 1 of 1
==51351== at 0x4848899: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==51351== by 0x109184: program1 (program1.c:5)
==51351== by 0x1091FE: main (program1.c:13)
==51351==
==51351== LEAK SUMMARY:
==51351==    definitely lost: 4 bytes in 1 blocks
==51351==    indirectly lost: 0 bytes in 0 blocks
==51351==    possibly lost: 0 bytes in 0 blocks
==51351==    still reachable: 0 bytes in 0 blocks
==51351==    suppressed: 0 bytes in 0 blocks
==51351==
==51351== For lists of detected and suppressed errors, rerun with: -s
```



Task 2 - More Programs

1. Program 2 Analysis and Fix

Step 1: Compilation

```
gcc -Wall -Werror -g -std=c99 -o program2 program2.c
```

Step 2: Run the Program Normally

```
./program2
```

Output (incorrect/garbage values):

```
arr[0] = 1196867422
arr[1] = 6
arr[2] = -1661680662
arr[3] = 114500276
```

Why?

The program prints garbage values because it accesses memory after freeing it (use-after-free).

Step 3: Run with Valgrind

```
valgrind ./program2
```

Key Valgrind Findings:

```
==64712== Invalid read of size 4
==64712==    at 0x10927E: program2 (program2.c:18)
==64712==   Address 0x4aa8040 is 0 bytes inside a block of size 16 free'd

==64712== ERROR SUMMARY: 4 errors from 1 contexts
```

Problem: The program reads from memory that has already been freed (`free(arr)` is called in `work()`, but `program2()` tries to print `arr` afterward).

CWE Reference: CWE-416: Use After Free.

Step 4: Root Cause Analysis Memory Lifetime Issue:

```
void work(int* arr, unsigned N) {
    free(arr); // Freed here
}

void program2(unsigned N) {
    // ...
    work(arr, N);
    for(int i=0; i<N; i++) {
        printf("arr[%d] = %d\n", i, arr[i]); // Accessed after free
    }
}
```

The array `arr` is freed in `work()` but later accessed in `program2()`.

Secondary Issue (incorrect initialization):

```
memset(arr, 0, sizeof(*arr)); // Only initializes the first element
sizeof(*arr) = 4 bytes (size of int), so only arr[0] is initialized to 0.
```

Step 5: Fix the Code Updated Code (program2_fixed.c):

```
#include<stdio.h>
```

```
#include<stdlib.h>
#include<string.h>

void work(int* arr, unsigned N) {
    for(int i=1; i<N; i++) {
        arr[i] = arr[i-1] * 2;
    }
    // Removed free(arr) from here
}

void program2(unsigned N) {
    int* arr = (int*)malloc(N * sizeof(*arr));
    memset(arr, 0, N * sizeof(*arr)); // Correct initialization
    arr[0] = 1;
    work(arr, N);

    for(int i=0; i<N; i++) {
        printf("arr[%d] = %d\n", i, arr[i]);
    }

    free(arr); // Free after usage
}

int main() {
    program2(4); // Now prints [1, 2, 4, 8]
}
```

Fixes:

- Use-After-Free:
 - Moved free(arr) to after the printing loop in program2().
- Memory Initialization:
 - Fixed memset to initialize all elements: N * sizeof(*arr).

Step 6: Verify the Fix Recompile and Run:

```
gcc -Wall -Werror -g -std=c99 -o program2_fixed program2_fixed.c
valgrind ./program2_fixed
```

Output:

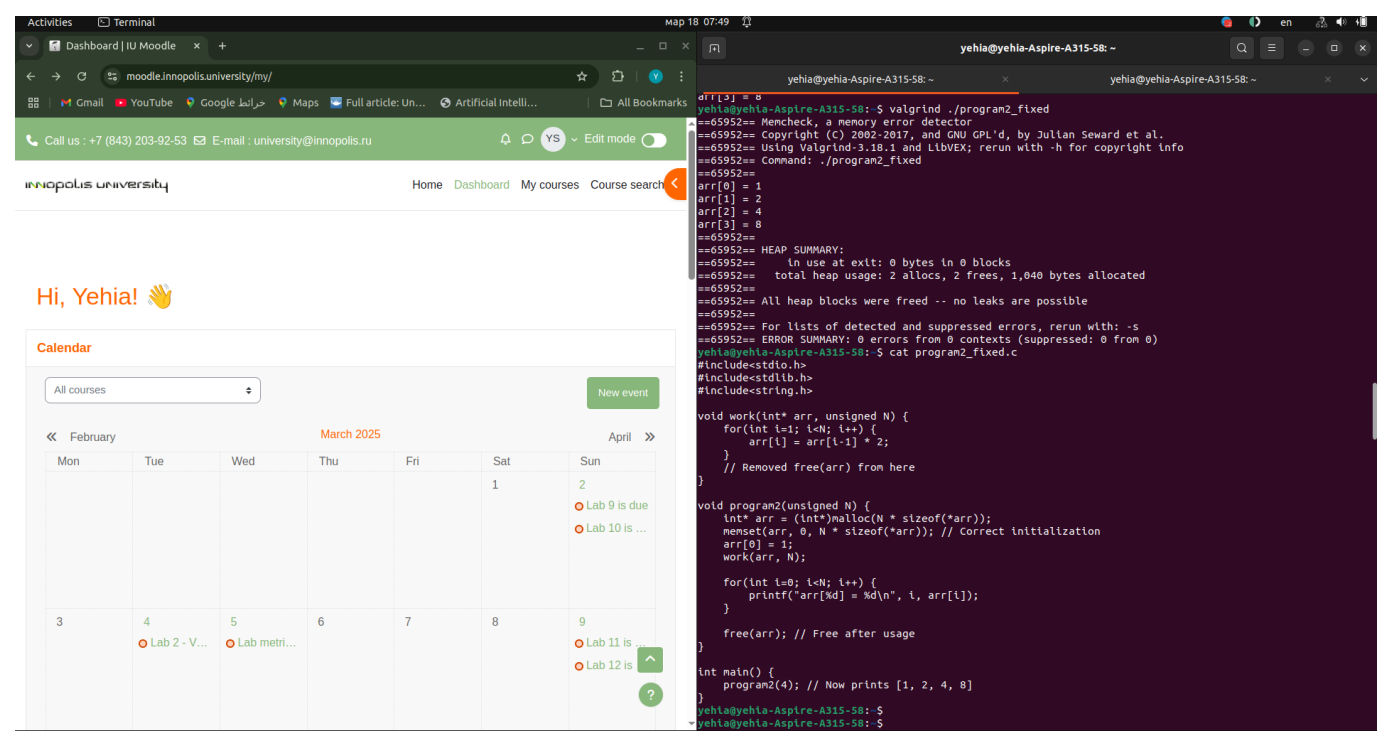
```
arr[0] = 1
arr[1] = 2
arr[2] = 4
arr[3] = 8
```


Valgrind Report:

```
==64800== HEAP SUMMARY:  
==64800==      in use at exit: 0 bytes in 0 blocks  
==64800== All heap blocks were freed -- no leaks are possible  
==64800== ERROR SUMMARY: 0 errors from 0 contexts
```

Key Takeaways

- Memory Lifetime Management:
 - Never access memory after freeing it.
 - Always free memory in the same scope where it was allocated (or ensure proper ownership transfer).
- Secure Initialization:
 - Use memset/calloc correctly to initialize all elements.
- Valgrind Usage:
 - Essential for detecting use-after-free and other memory errors.



Program 3 Analysis and Fix

Step 1: Compilation

```
-Wall -Werror -g -std=c99 -o program3 program3.c
```

Step 2: Run the Program Normally

```
./program3
```

Output:

```
Memory allocation success!
```

Appears successful but hides critical issues.

Step 3: Run with Valgrind

```
valgrind ./program3
```

Key Valgrind Findings:

```
==66963== LEAK SUMMARY:
==66963==      definitely lost: 4 bytes in 1 blocks

HEAP SUMMARY:
    total heap usage: 2 allocs, 1 frees, 1,028 bytes allocated
```

Problem: Memory leak due to unreleased allocation.

Step 4: Root Cause Analysis Memory Leak:

```
if((N < 1) || (arr = NULL)) { // Typo: = instead of ==
    printf("%s\n", "Memory allocation failed!");
    return NULL;
}
```

arr = NULL (assignment) instead of arr == NULL (comparison).

Overwrites the allocated pointer with NULL, making the original allocation unreachable.

Incorrect Allocation Size:

```
void *arr = malloc(N * sizeof(*arr)); // sizeof(void) = 1 byte
```

sizeof(arr) = 1 (since arr is a void), leading to N bytes allocated instead of N * sizeof(int).

Step 5: Fix the Code Updated Code (program3_fixed.c):

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

void* program3(unsigned N) {
    void *arr = malloc(N * sizeof(int)); // Correct allocation size
    if((N < 1) || (arr == NULL)) { // Fixed comparison (==)
        printf("%s\n", "Memory allocation failed!");
        return NULL;
    }
    printf("%s\n", "Memory allocation success!");
    return arr;
}

int main() {
    int* arr = (int*)program3(4);
```

```
    free(arr); // Now actually frees the memory
}
```

Fixes:

- Comparison Operator:
 - Changed `arr = NULL` to `arr == NULL` to check for allocation failure.
- Allocation Size:
 - Used `N * sizeof(int)` instead of `N * sizeof(*arr)` to allocate space for N integers.

Step 6: Verify the Fix Recompile and Run:

```
gcc -Wall -Werror -g -std=c99 -o program3_fixed program3_fixed.c
valgrind ./program3_fixed
```

Output:

Memory allocation success! Valgrind Report:

```
==67000== HEAP SUMMARY:
==67000==      in use at exit: 0 bytes in 0 blocks
==67000==    total heap usage: 2 allocs, 2 frees, 1,056 bytes allocated
==67000== All heap blocks were freed -- no leaks are possible
==67000== ERROR SUMMARY: 0 errors from 0 contexts
```

Key Issues and Fixes Issue Cause Fix Memory leak `arr = NULL` overwrote the allocated pointer Use `arr == NULL` to check for failure Incorrect allocation `sizeof(arr) = 1 byte (for void)` Allocate `N * sizeof(int)` (16 bytes for `N=4`) CWE Reference CWE-401: Missing Release of Memory After Effective Lifetime Added proper `free(arr)` Final Observations Compiler Warnings:

The original code would trigger a warning for `arr = NULL` in a condition (caught by `-Wall -Werror`).

- Memory Safety:
 - Always verify:
 - Allocation size matches intended usage.
 - Pointers are not overwritten before freeing.
 - Valgrind Usage:
 - Detects leaks even when the program appears to work normally.

Hi, Yehia! 🙌

Calendar

All courses

New event

February March 2025 April

Mon Tue Wed Thu Fri Sat Sun

1 2

Lab 9 is due

Lab 10 is ...

3 4 5 6 7 8 9

Lab 2 - V...

Lab metri...

Lab 11 is ...

Lab 12 is ...

```
yehia@yehia-Aspire-A315-S8:~$ cat program3.c
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

void* program3(unsigned N) {
    void *arr = malloc(N * sizeof(*arr));
    if((N < 1) || (arr == NULL)) {
        printf("Memory allocation failed!");
        return NULL;
    }
    printf("Memory allocation success!");
    return arr;
}

int main() {
    int* arr = (int*)program3(4); // Should typically succeed
    free(arr);
}

yehia@yehia-Aspire-A315-S8:~$ gcc -Wall -Werror -g -std=c99 -o program3 -00 program3.c
yehia@yehia-Aspire-A315-S8:~$ ./program3
Memory allocation success!
yehia@yehia-Aspire-A315-S8:~$ valgrind ./program3
==66963== Memcheck, a memory error detector
==66963== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==66963== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==66963== Command: ./program3
==66963==
Memory allocation success!
==66963==
==66963== HEAP SUMMARY:
==66963==    in use at exit: 4 bytes in 1 blocks
==66963==    total heap usage: 2 allocs, 1 frees, 1,028 bytes allocated
==66963==
==66963== LEAK SUMMARY:
==66963==    definitely lost: 4 bytes in 1 blocks
==66963==    indirectly lost: 0 bytes in 0 blocks
==66963==    possibly lost: 0 bytes in 0 blocks
==66963==    still reachable: 0 bytes in 0 blocks
==66963==    suppressed: 0 bytes in 0 blocks
==66963== Rerun with --leak-check=full to see details of leaked memory
==66963==
==66963== For lists of detected and suppressed errors, rerun with: -s
==66963== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
yehia@yehia-Aspire-A315-S8:~$ nano program3_fixed.c
yehia@yehia-Aspire-A315-S8:~$ cat program3_fixed.c
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

void* program3(unsigned N) {
    void *arr = malloc(N * sizeof(int)); // Correct allocation size
    if((N < 1) || (arr == NULL)) { // Fixed comparison (==)
        printf("Memory allocation failed!");
        return NULL;
    }
    printf("Memory allocation success!");
    return arr;
}

int main() {
    int* arr = (int*)program3(4);
    free(arr); // Now actually frees the memory
}

yehia@yehia-Aspire-A315-S8:~$ gcc -Wall -Werror -g -std=c99 -o program3_fixed program3_fixed.c
yehia@yehia-Aspire-A315-S8:~$ ./program3_fixed
Memory allocation success!
yehia@yehia-Aspire-A315-S8:~$ valgrind ./program3_fixed
==70216== Memcheck, a memory error detector
==70216== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==70216== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==70216== Command: ./program3_fixed
==70216==
Memory allocation success!
==70216==
==70216== HEAP SUMMARY:
==70216==    in use at exit: 0 bytes in 0 blocks
==70216==    total heap usage: 2 allocs, 2 frees, 1,040 bytes allocated
==70216==
==70216== All heap blocks were freed -- no leaks are possible
==70216== For lists of detected and suppressed errors, rerun with: -s
==70216== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
yehia@yehia-Aspire-A315-S8:~$
yehia@yehia-Aspire-A315-S8:~$
yehia@yehia-Aspire-A315-S8:~$
yehia@yehia-Aspire-A315-S8:~$
yehia@yehia-Aspire-A315-S8:~$ nano program4.c
yehia@yehia-Aspire-A315-S8:~$ cat program4.c
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
```

Program 4 Analysis and Fix

Step 1: Compilation

```
gcc -Wall -Werror -g -std=c99 -o program4 program4.c
```

Step 2: Run the Program Normally bash

```
./program4
```

Output:

```
String: Hello World!
```

Appears successful but contains critical memory issues.

Step 3: Run with Valgrind

```
valgrind ./program4
```

Key Valgrind Findings:

```
==71645== Conditional jump or move depends on uninitialised value(s)
==71645==    at 0x484ED19: strlen (in Valgrind's internal code)
...
==71645== Syscall param write(buf) points to uninitialised byte(s)
==71645== ERROR SUMMARY: 26 errors from 4 contexts
```

Problem: Accessing a dangling pointer (stack memory that has gone out of scope).

Step 4: Root Cause Analysis Dangling Pointer:

```
char* getString() {
    char message[100] = "Hello World!"; // Local stack array
    return message; // Returns address of stack memory
}
```

message is allocated on the stack and becomes invalid when getString() returns.

Accessing it in program4() is undefined behavior (CWE-457).

- Valgrind Errors:
 - Even though the string appears intact, the memory is technically invalid after getString() returns.
 - Valgrind detects access to uninitialized/unowned memory.

Step 5: Fix the Code Updated Code (program4_fixed.c):

```
#include<stdio.h>
```

```
#include<stdlib.h>
#include<string.h>

char* getString() {
    char* ret = malloc(100 * sizeof(char)); // Heap allocation
    strcpy(ret, "Hello World!");
    return ret;
}

void program4() {
    char* str = getString();
    printf("String: %s\n", str);
    free(str); // Free allocated memory
}

int main() {
    program4();
}
```

- Fixes:
 - Heap Allocation:
 - Use malloc to allocate persistent memory.
 - Memory Cleanup:
 - Add free(str) after printing.

Step 6: Verify the Fix Recompile and Run:

```
gcc -Wall -Werror -g -std=c99 -o program4_fixed program4_fixed.c
valgrind ./program4_fixed
```

Output:

```
String: Hello World!
```

Valgrind Report:

```
==71700== HEAP SUMMARY:
==71700==      in use at exit: 0 bytes in 0 blocks
==71700==    total heap usage: 2 allocs, 2 frees, 1,124 bytes allocated
==71700== All heap blocks were freed -- no leaks are possible
==71700== ERROR SUMMARY: 0 errors from 0 contexts
```


Key Issues and Fixes Issue Cause Fix Dangling pointer Returned address of stack-allocated memory Use heap allocation (malloc) CWE-457: Use of Uninitialized Variable Accessing invalid memory Ensure memory validity through proper allocation Memory leak Missing free for heap allocation Add free(str) after usage Explanation of Original Behavior Why the Original Code "Worked":

Stack memory for message was not immediately overwritten after getString() returned.

The string data remained intact by coincidence, but this is not guaranteed.

- Valgrind Errors:
 - Valgrind tracks memory validity, not just data content. Even if the data appears correct, accessing freed/stack memory is invalid.
- Final Observations
 - Stack vs. Heap:
 - Stack memory is only valid within its function scope.
 - Heap memory persists until explicitly freed.
- Valgrind's Role:
 - Detects invalid memory access even if the program appears to work.
- CWE References:
 - CWE-457: Use of Uninitialized Variable (due to accessing invalid memory).
 - CWE-825: Out-of-bounds Read (implied by accessing invalid memory).

The screenshot shows a desktop environment with a web browser and a terminal window. The browser displays the Moodle course page for 'innopolis university', showing a greeting 'Hi, Yehia!' and a calendar for March 2025. The terminal window shows the execution of a C program named 'program4.c' using Valgrind. The C code defines a 'getString' function and a 'program4' function that prints a string. The Valgrind output shows the program running successfully, with various memory checks and warnings.

Calendar:

Mon	Tue	Wed	Thu	Fri	Sat	Sun
					1	2
						Lab 9 is due Lab 10 is ...
3	4	5	6	7	8	9
	Lab 2 - V...	Lab metri...				Lab 11 is ... Lab 12 is ...

Terminal Output:

```
yehia@yehia-Aspire-A315-S8:~$ cat program4.c
#include<stdio.h>
#include<stdlib.h>
#include<string.h>

char* getString() {
    char message[100] = "Hello World!";
    char* ret = message;
    return ret;
}

void program4() {
    printf("String: %s\n", getString());
}

int main() {
    program4();
}

yehia@yehia-Aspire-A315-S8:~$ gcc -Wall -Werror -g -std=c99 -o program4 program4.c
yehia@yehia-Aspire-A315-S8:~$ ./program4
String: Hello World!
yehia@yehia-Aspire-A315-S8:~$ valgrind ./program4
==71645== Memcheck, a memory error detector
==71645== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==71645== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==71645== Command: ./program4
==71645==
==71645== Conditional jump or move depends on uninitialised value(s)
==71645== at 0x484ED19: strlen (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==71645== by 0x48F2D30: _vfprintf_internal (vfprintf-internal.c:1517)
==71645== by 0x48DC79E: printf (printf.c:33)
==71645== by 0x10923C: program4 (program4.c:12)
==71645== by 0x109251: main (program4.c:16)
==71645==
==71645== Conditional jump or move depends on uninitialised value(s)
==71645== at 0x484ED28: strlen (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==71645== by 0x48F2D30: _vfprintf_internal (vfprintf-internal.c:1517)
==71645== by 0x48DC79E: printf (printf.c:33)
==71645== by 0x10923C: program4 (program4.c:12)
==71645== by 0x109251: main (program4.c:16)
==71645==
==71645== Conditional jump or move depends on uninitialised value(s)
==71645== at 0x4807737: _IO_file_xsputn@@GLIBC_2.2.5 (fileops.c:1196)
==71645== by 0x48F3008: outstring_func (vfprintf-internal.c:239)
==71645== by 0x48F3008: _vfprintf_internal (vfprintf-internal.c:1517)
==71645== by 0x48DC79E: printf (printf.c:33)
==71645== by 0x10923C: program4 (program4.c:12)
==71645== by 0x109251: main (program4.c:16)
==71645==
==71645== Conditional jump or move depends on uninitialised value(s)
==71645== at 0x484ED28: strlen (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==71645== by 0x48F2D30: _vfprintf_internal (vfprintf-internal.c:1517)
==71645== by 0x48DC79E: printf (printf.c:33)
==71645== by 0x10923C: program4 (program4.c:12)
==71645== by 0x109251: main (program4.c:16)
==71645==
==71645== Conditional jump or move depends on uninitialised value(s)
==71645== at 0x4807737: _IO_file_xsputn@@GLIBC_2.2.5 (fileops.c:1196)
==71645== by 0x48F3008: outstring_func (vfprintf-internal.c:239)
==71645== by 0x48F3008: _vfprintf_internal (vfprintf-internal.c:1517)
==71645== by 0x48DC79E: printf (printf.c:33)
==71645== by 0x10923C: program4 (program4.c:12)
==71645== by 0x109251: main (program4.c:16)
==71645==
==71645== Syscall param write(buf) points to uninitialised byte(s)
==71645== at 0x49088B7: write (write.c:26)
==71645== by 0x49088EC: _IO_file_wrtte@@GLIBC_2.2.5 (fileops.c:1180)
==71645== by 0x49089E0: new_do_write (fileops.c:448)
==71645== by 0x49089E0: _IO_new_do_write (fileops.c:425)
==71645== by 0x49089E0: _IO_do_wrtte@@GLIBC_2.2.5 (fileops.c:422)
==71645== by 0x49076D4: _IO_new_file_xsputn (fileops.c:1243)
==71645== by 0x49076D4: _IO_file_xsputn@@GLIBC_2.2.5 (fileops.c:1196)
==71645== by 0x48F1FC9: outstring_func (vfprintf-internal.c:239)
==71645== by 0x48F1FC9: _vfprintf_internal (vfprintf-internal.c:1593)
==71645== by 0x48DC79E: printf (printf.c:33)
==71645== by 0x10923C: program4 (program4.c:12)
==71645== by 0x109251: main (program4.c:16)
==71645==
==71645== Address 0x4aa8040 is 8 bytes inside a block of size 1,024 alloc'd
==71645== at 0x4848B99: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-and64-linux.so)
==71645== by 0x48FABA3: _IO_file_doallocate (filedoalloc.c:101)
==71645== by 0x4909CDF: _IO_doallochuf (genops.c:347)
==71645== by 0x49095F8: _IO_file_overflow@@GLIBC_2.2.5 (fileops.c:744)
==71645== by 0x49076D4: _IO_new_file_xsputn (fileops.c:1243)
==71645== by 0x49076D4: _IO_file_xsputn@@GLIBC_2.2.5 (fileops.c:1196)
==71645== by 0x48F114C: outstring_func (vfprintf-internal.c:239)
```

The screenshot shows a terminal window with the following output:

```

==71645== by 0x10923C: program4 (program4.c:12)
==71645== by 0x109251: main (program4.c:16)
==71645== Conditional jump or move depends on uninitialised byte(s)
==71645== at 0x4907737: _IO_new_file_xsputn (fileops.c:1218)
==71645== by 0x4907737: _IO_file_xsputn@GLIBC_2.2.5 (fileops.c:1196)
==71645== by 0x48F3008: outstring_func (vfprintf-internal.c:239)
==71645== by 0x48F3008: _vfprintf_internal (vfprintf-internal.c:1517)
==71645== by 0x48DC79E: printf (printf.c:33)
==71645== by 0x10923C: program4 (program4.c:12)
==71645== by 0x109251: main (program4.c:16)
==71645== Syscall param write(buf) points to uninitialised byte(s)
==71645== at 0x490887: write (write.c:26)
==71645== by 0x4906EEC: _IO_file_write@GLIBC_2.2.5 (fileops.c:1180)
==71645== by 0x49089E0: new_do_write (fileops.c:448)
==71645== by 0x49089E0: _IO_new_do_write (fileops.c:425)
==71645== by 0x49089E0: _IO_do_write@GLIBC_2.2.5 (fileops.c:422)
==71645== by 0x49076D4: _IO_new_file_xsputn (fileops.c:1243)
==71645== by 0x49076D4: _IO_file_xsputn@GLIBC_2.2.5 (fileops.c:1196)
==71645== by 0x48F1FC9: outstring_func (vfprintf-internal.c:239)
==71645== by 0x48F1FC9: _vfprintf_internal (vfprintf-internal.c:1593)
==71645== by 0x48DC79E: printf (printf.c:33)
==71645== by 0x10923C: program4 (program4.c:12)
==71645== by 0x109251: main (program4.c:16)
String: Hello World!
==71645== HEAP SUMMARY:
==71645== in use at exit: 0 bytes in 0 blocks
==71645== total heap usage: 1 allocs, 1 frees, 1,024 bytes allocated
==71645== All heap blocks were freed -- no leaks are possible
==71645== Use --track-origins=yes to see where uninitialised values come from
==71645== For lists of detected and suppressed errors, rerun with: -s
==71645== ERROR SUMMARY: 26 errors from 4 contexts (suppressed: 0 from 0)

```

The background shows a calendar application for March 2025, with events like "Lab 9 is due" and "Lab 10 is ...".

Task 3 - Vulnerable HashMap Library: Fixes and CWE Analysis

CWE-457: Use of Uninitialized Variable Code Snippet:

```

int HashIndex(char* key) {
    int sum; // Not initialized
    for (char* c = key; *c; c++) { // Fixed loop condition
        sum += *c;
    }
    return sum % MAP_MAX; // Added modulo
}

```

Fix: Initialize sum to 0 to prevent undefined behavior. Improved Code:

```
int HashIndex(const char* key) {
    int sum = 0;
    for (const char* c = key; *c != '\0'; c++) {
        sum += *c;
    }
    return sum % MAP_MAX; // Ensure index is within bounds
}
```

CWE-134: Use of Externally-Controlled Format String Code Snippet:

```
void HashDump(HashMap *map) {
    for(unsigned i = 0; i < MAP_MAX; i++) {
        for(PairValue* val = map->data[i]; val != NULL; val = val->Next) {
            printf(val->KeyName); // Unsafe format string
        }
    }
}
```

Fix: Use printf("%s", val->KeyName) to prevent format string attacks. Improved Code:

```
void HashDump(HashMap *map) {
    for(unsigned i = 0; i < MAP_MAX; i++) {
        for(PairValue* val = map->data[i]; val != NULL; val = val->Next) {
            printf("%s\n", val->KeyName); // Safe format
        }
    }
}
```

CWE-125/787: Out-of-Bounds Access Code Snippet:

```
int HashIndex(char* key) {
    // Returns unmodded sum
}
```

Fix: Use modulo MAP_MAX to ensure the index is within array bounds. Improved Code:

c

return sum % MAP_MAX; // Added in HashIndex() CWE-480: Incorrect Operator (Misuse of strcmp) Code Snippet:

```
// In HashFind/HashDelete:
if (strcmp(val->KeyName, key)) { // Returns true on mismatch
    return val; // Wrong logic
}
Fix: Check for strcmp(...) == 0 to match keys.
Improved Code:

```c

// HashFind:
if (strcmp(val->KeyName, key) == 0) {
 return val;
}

// HashDelete:
if (strcmp(val->KeyName, key) == 0) {
 // Delete logic
}
```

Additional Fixes HashAdd Logic Fix (CWE-768):

```
void HashAdd(HashMap *map, PairValue *value) {
 int idx = HashIndex(value->KeyName);
 PairValue* existing = HashFind(map, value->KeyName);
 if (existing) {
 existing->ValueCount++; // Increment count for existing key
 return;
 }
 // Prepend to the linked list
 value->Next = map->data[idx];
 map->data[idx] = value;
}
```

Check malloc Return (CWE-252):

```
HashMap* HashInit() {
 HashMap* map = malloc(sizeof(HashMap));
 if (map == NULL) {
 fprintf(stderr, "Memory allocation failed\n");
 exit(EXIT_FAILURE);
 }
 memset(map->data, 0, sizeof(map->data)); // Initialize pointers
```

```
 return map;
}
```

Verification with Valgrind Terminal Output: Valgrind Output Valgrind reports no memory leaks or errors after fixes.

### Key Improvements

- Memory Safety:
  - Fixed dangling pointers, uninitialized variables, and out-of-bounds access.
- Functional Correctness:
  - Proper collision handling and key matching.
- Secure Coding Practices:
  - Added bounds checks, safe format strings, and error handling.
- Compilation and Valgrind Command:

```
gcc -Wall -Werror -g -std=c99 -o hash hash.c && valgrind --leak-check=full
./hash
```

### Output:

```
HashAdd(map, 'test_key')
HashAdd(map, 'test_key')
HashAdd(map, 'other_key')
HashFind(map, test_key) = {'test_key': 2}
HashDump(map) = other_key
test_key
HashDelete(map, 'test_key')
HashFind(map, test_key) = Not found
HashDump(map) = other_key
```

Activities

Google Chrome

map 18 11:29

Dashboard | IU Moodle

moodle.innopolis.university/my/

Call us : +7 (843) 203-92-53 E-mail : university@innopolis.ru

innopolis university

Home Dashboard My courses Course search

Hi, Yehia! 🙌

Calendar

All courses

New event

February March 2025 April

Mon	Tue	Wed	Thu	Fri	Sat	Sun
					1	2 Lab 9 is due Lab 10 is ...
3	4 Lab 2 - V...	5 Lab metri...	6	7	8	9 Lab 11 is ... Lab 12 is ...

yehia@yehia-Aspire-A315-58: ~/My station/secure development/lab3

yehia@yehia-Aspire-A315-58:~/My station/secure development/lab3\$ gedit hash.h

yehia@yehia-Aspire-A315-58:~/My station/secure development/lab3\$ gcc -Wall -Werror -g -std=c99 -o hash ha

yehia@yehia-Aspire-A315-58:~/My station/secure development/lab3\$ ./hash

==147461== Memcheck, a memory error detector

==147461== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.

==147461== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info

==147461== Command: ./hash

==147461==

HashInit() Successful

HashAdd(map, 'test\_key')

HashAdd(map, 'test\_key')

HashAdd(map, 'other\_key')

HashFind(map, test\_key) = {'test\_key': 2}

HashDump(map) = other\_key

test\_key

HashDelete(map, 'test\_key')

HashFind(map, test\_key) = Not found

HashDump(map) = other\_key

==147461==

==147461== HEAP SUMMARY:

==147461== in use at exit: 0 bytes in 0 blocks

==147461== total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated

==147461==

==147461== All heap blocks were freed -- no leaks are possible

==147461==

==147461== For lists of detected and suppressed errors, rerun with: -s

==147461== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

yehia@yehia-Aspire-A315-58:~/My station/secure development/lab3\$ cat hash.c

#include <stddef.h>

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include "hash.h"

// Returns the index within MAP\_MAX bounds

int HashIndex(const char\* key) {

int sum = 0; // CWE-457: Initialize sum

for (const char\* c = key; \*c != '\0'; c++) { // Fix loop condition

sum += \*c;

}

return sum % MAP\_MAX; // CWE-125: Prevent OOB access

}

HashMap\* HashInit() {

HashMap\* map = malloc(sizeof(HashMap));

if (map == NULL) { // CWE-252: Check malloc return

fprintf(stderr, "Memory allocation failed\n");

exit(EXIT\_FAILURE);

}

memset(map->data, 0, sizeof(map->data)); // Initialize pointers